

Produto interno e matrizes em C

Relatório técnico de implementação, entrada/saída, desempenho e código de máquina

Integrantes: Vitória Eloise, Gustavo de Moraes e Anna Letícia.
Base documental: repositório Projeto_Implementacao_Produto_Interno [1].
Consulta: 21 de setembro de 2026. Commit: a88300a1bf3ef69351fd34d373347b8bbfec31b8.

Resumo

Este relatório examina uma implementação sequencial de multiplicação de matrizes em C, com armazenamento contíguo, alinhamento de 64 bytes e laços na ordem $i \rightarrow k \rightarrow j$. São relacionados o código-fonte, os alvos de compilação do GCC, as medições publicadas e os trechos de código de máquina disponíveis. Embora o projeto mencione produto interno, o programa calcula uma matriz $C = AB$; cada elemento de C corresponde ao produto interno de uma linha de A com uma coluna de B.

Na máquina de Vitória Eloise, os alvos otimizados apresentam ganhos expressivos em relação a base. O maior valor publicado é 6,7186 GFLOPS, com o3 e ordem 1.000. Entretanto, fast fica abaixo de o2 e o3 nas três ordens medidas. Os arquivos assembly revelam FMA escalar no fast de Eloise e FMA vetorial de 256 bits no fast de Gustavo, diferença que impede tratar o nome do alvo como garantia de uma estratégia única de otimização.

1. Escopo e evidências disponíveis

Material	Eloise	Gustavo	Letícia
Especificações de CPU, RAM e GCC	Disponíveis	Disponíveis	Ausentes
Assembly base e fast	Disponível	Disponível	Ausente
Tempos e GFLOPS: 3 ordens × 4 alvos	12 registros	Ausentes	Ausentes
Carga/escrita separadas: CSV e BIN	Ausentes	Ausentes	Ausentes

O arquivo benchmark_resultado.txt repete os mesmos 12 resultados do CSV de Eloise; ele não constitui uma segunda máquina ou nova repetição experimental. As pastas de Letícia contêm apenas arquivos README vazios na versão consultada. Não foram produzidos benchmarks em outra máquina para substituir os dados dos integrantes.

Assim, a comparação experimental entre máquinas e o comparativo completo de E/S permanecem parciais. As tabelas e figuras distinguem medidas observadas de dados não disponíveis (ND). Especificações nominais complementadas pelos fabricantes são identificadas nas referências; não são frequências medidas durante a execução.

2. Implementação e acesso à memória

2.1 Armazenamento linear e alinhamento

Em `src/matmul.c`, `A` tem dimensões $M \times K$, `B` tem $K \times N$ e `C` tem $M \times N$. Cada matriz ocupa um único bloco de `float`, organizado por linhas (row-major). Os acessos `A[i*K+k]`, `B[k*N+j]` e `C[i*N+j]` substituem uma estrutura de ponteiros por linha e tornam explícita a posição linear de cada elemento. Para `float` de 4 bytes, o deslocamento do elemento (i,j) de `C` é $4(iN+j)$ bytes.

```
posix_memalign((void **)&A, 64, M * K * sizeof(float));
/* B e C recebem alocações independentes de 64 bytes. */
float * restrict ptr_A = (float *)__builtin_assume_aligned(A, 64);
float * restrict ptr_B = (float *)__builtin_assume_aligned(B, 64);
float * restrict ptr_C = (float *)__builtin_assume_aligned(C, 64);
```

O fragmento resume a alocação; no original, os retornos de `posix_memalign` são verificados. A função garante o alinhamento inicial; `__builtin_assume_aligned` comunica essa propriedade ao GCC, sem realocar ou testar o ponteiro [5]. Os qualificadores `restrict` expressam a ausência de sobreposição relevante entre os blocos e podem favorecer otimizações de acesso.

Alinhar a base não alinha automaticamente cada linha a 64 bytes: isso depende de $4N$ ser múltiplo de 64. Para $N = 1.000$, cada linha tem 4.000 bytes e o início alterna entre offsets 0 e 32 módulo 64; para 2.000 e 4.000, o passo entre linhas é múltiplo de 64. Nos três casos, os passos são múltiplos de 32 bytes. O alinhamento favorece acessos regulares, mas não garante vetorização.

2.2 Justificativa da ordem $i \rightarrow k \rightarrow j$

```
for (size_t i = 0; i < M; i++) {
    for (size_t k = 0; k < K; k++) {
        float r = ptr_A[i * K + k];
        for (size_t j = 0; j < N; j++) {
            ptr_C[i * N + j] += r * ptr_B[k * N + j];
        }
    }
}
```

Com i e k fixos, r é reutilizado ao longo de j , enquanto `B` e `C` são percorridas sequencialmente. A ordem usual $i \rightarrow j \rightarrow k$ acessaria `B` por colunas, com salto de N elementos entre leituras consecutivas. A ordem adotada melhora a localidade espacial e permite processar várias colunas independentes em uma instrução SIMD.

A complexidade permanece $O(MKN)$, ou $O(n^3)$ para matrizes quadradas. Não há blocagem de cache, transposição, OpenMP nem chamadas a BLAS. O programa é sequencial: múltiplos núcleos disponíveis não implicam seu uso simultâneo. Para $n = 4.000$, `A`, `B` e `C` ocupam juntas 192 MB decimais (aproximadamente 183,1 MiB), sem contar outras estruturas.

3. Compilação e medidas de cálculo

Alvo	Opções específicas do Makefile
base	-O0 -g0
o2	-O2
o3	-O3
fast	-O3 -march=native -ffast-math -funroll-loops

Todos usam gcc, C99, -Wall, -Wextra e -D_GNU_SOURCE. O alvo fast combina seleção de instruções locais, relaxamento de regras de ponto flutuante e tentativa de desenrolamento dos laços [4, 6]. Não é um sinônimo literal de -Ofast. As opções podem alterar a geração de código sem assegurar ganho de desempenho.

3.1 O que o cronômetro mede

CLOCK_MONOTONIC delimita a leitura binária, o núcleo de cálculo e a escrita binária. TEMPO_IO soma abertura, leitura, alocação dos três blocos e fechamento dos arquivos de entrada à abertura, escrita e fechamento da saída. A inicialização de C com zeros fica fora tanto de TEMPO_COMP quanto de TEMPO_IO. Portanto, somar esses dois campos não reproduz o tempo total do processo.

GFLOPS = 2MKN / (10⁹ × TEMPO_COMP). A contagem atribui uma multiplicação e uma adição a cada atualização. Para n = 1.000, 2.000 e 4.000, são respectivamente 2, 16 e 128 bilhões de operações. Os 12 GFLOPS publicados foram recalculados e conferem com os tempos, dentro do arredondamento de quatro casas decimais.

3.2 Resultados de Vitória Eloise

Ordem n	Alvo	Cálculo (s)	GFLOPS	Ganho vs. base
1.000	base	2,629554	0,7606	1,00×
1.000	o2	0,298990	6,6892	8,79×
1.000	o3	0,297682	6,7186	8,83×
1.000	fast	0,322893	6,1940	8,14×
2.000	base	20,782665	0,7699	1,00×
2.000	o2	2,718444	5,8857	7,65×
2.000	o3	2,775273	5,7652	7,49×
2.000	fast	2,962804	5,4003	7,01×
4.000	base	165,193151	0,7749	1,00×
4.000	o2	21,262834	6,0199	7,77×
4.000	o3	21,208064	6,0354	7,79×
4.000	fast	24,555785	5,2126	6,73×

Fonte: `maquinas/eloise/benchmark_eloise.csv` [1]. Ganho = tempo de base / tempo do alvo, na mesma ordem. Há um registro por combinação; não estão documentados número de repetições, dispersão, aquecimento ou ordem completa das execuções. Diferenças pequenas entre o2 e o3 não sustentam conclusões estatísticas.

4. Entrada e saída: CSV e binário

O CSV possui cabeçalho M,N e valores textuais. csv2bin.c lê os valores com fscanf e escreve cabeçalho e dados em um buffer binário; bin2csv.c usa fread e formata cada valor com fprintf("%.6f"). O BIN armazena dois uint64_t e uma sequência de float. Nas plataformas registradas, o tamanho esperado de uma matriz quadrada é $16 + 4n^2$ bytes: 4.000.016, 16.000.016 e 64.000.016 bytes.

O processamento textual exige conversão entre texto e números; o binário transfere a representação em memória. Isso oferece uma justificativa técnica para investigar seu custo, mas o repositório não contém tempos isolados das quatro operações solicitadas. Cronometrar o conversor inteiro também não separaria leitura de um formato e escrita do outro.

Ordem n	Carga CSV (s)	Escrita CSV (s)	Carga BIN (s)	Escrita BIN (s)	E/S BIN total fast (s)
1.000	ND	ND	ND	ND	0,005034
2.000	ND	ND	ND	ND	0,025277
4.000	ND	ND	ND	ND	0,106282

ND = não disponível. A última coluna é uma medida agregada: lê A e B, aloca A/B/C e escreve C. Ela não pode ser dividida por três para inferir tempos individuais.

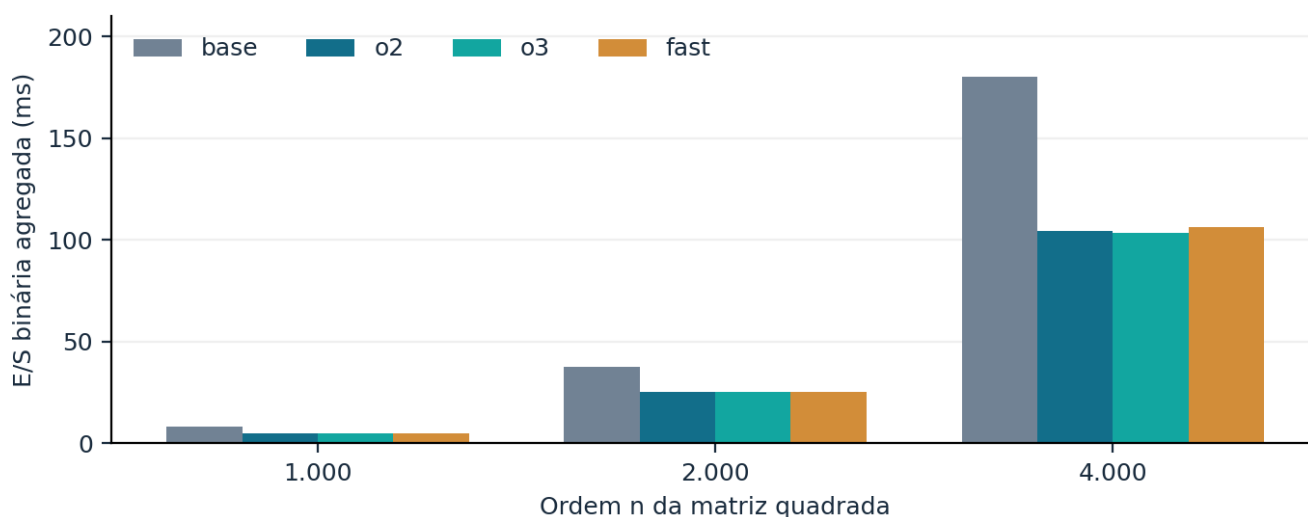


Figura 1. E/S binária agregada observada na máquina de Eloise, por ordem e alvo. Não há séries CSV publicadas. Esta figura apresenta a evidência existente e não substitui o comparativo CSV × BIN solicitado. Fonte: [1].

Em $n = 4.000$, a E/S agregada varia de 0,103401 s (o3) a 0,180081 s (base). Como não há registro de controle de cache de arquivos ou de repetições, essa diferença não pode ser atribuída somente à otimização. fwrite e fclose não equivalem a uma confirmação de persistência física com fsync: os tempos também refletem o sistema de arquivos e o cache do sistema operacional.

Para completar a comparação, devem ser medidos separadamente carga CSV, escrita CSV, carga BIN e escrita BIN sobre as mesmas matrizes, declarando aquecimento, política de cache e repetições. O formato binário atual não normaliza endianness nem representação numérica; sua portabilidade depende de plataformas compatíveis.

5. Especificações das máquinas

A tabela combina os registros locais de CPU, RAM e GCC [1] com microarquitetura e frequências nominais dos fabricantes [2, 3, 7]. L1 e L2 são indicados por núcleo e no total; L3 é compartilhado. As capacidades de cache usam as unidades binárias dos registros locais.

Característica	Vitória Eloise	Gustavo de Moraes	Anna Letícia
Processador	AMD Ryzen 5 5500	Intel Core i5-11300H	ND
Microarquitetura	Zen 3 (Cezanne)	Willow Cove (Tiger Lake)	ND
Núcleos / threads	6 / 12	4 / 8	ND
Frequência base nominal	3,60 GHz	3,10 GHz (35 W) 2,60 GHz (28 W)	ND
Boost máximo nominal	Até 4,20 GHz	Até 4,40 GHz	ND
L1 dados por núcleo (total)	32 KiB (192 KiB)	48 KiB (192 KiB)	ND
L1 instruções por núcleo (total)	32 KiB (192 KiB)	32 KiB (128 KiB)	ND
L2 por núcleo (total)	512 KiB (3 MiB)	1,25 MiB (5 MiB)	ND
L3 compartilhado	16 MiB	8 MiB	ND
SSE4.2 / AVX / AVX2	Presentes	Presentes	ND
FMA	Presente	Presente	ND
AVX-512	Não consta nas flags	Presente nas flags	ND
RAM visível ao ambiente	7,7 GiB	7,6 GiB	ND
GCC registrado	15.2.0 (Ubuntu 15.2.0-16ubuntu1)	15.2.0 (Ubuntu 15.2.0-16ubuntu1)	ND
Virtualização registrada	Microsoft; full	Microsoft; full	ND

Frequência máxima não é frequência sustentada do benchmark. A RAM listada corresponde ao ambiente que executou free, podendo diferir da memória física instalada. Os registros indicam hipervisor Microsoft; não identificam, por si só, toda a configuração de virtualização.

5.1 Relação com o conjunto de trabalho

Uma matriz de ordem 1.000 ocupa aproximadamente 3,8 MiB; em 2.000, 15,3 MiB; e em 4.000, 61,0 MiB. B sozinha já supera o L3 de ambas as máquinas em 4.000. Em contraste, uma linha de C ocupa de 4 a 16 kB decimais. A ordem dos laços favorece a reutilização dessa linha, enquanto B precisa ser percorrida novamente a cada novo i. Essa relação sugere investigar blocagem; sem contadores de cache, não prova um gargalo específico.

6. Análise comparativa do desempenho

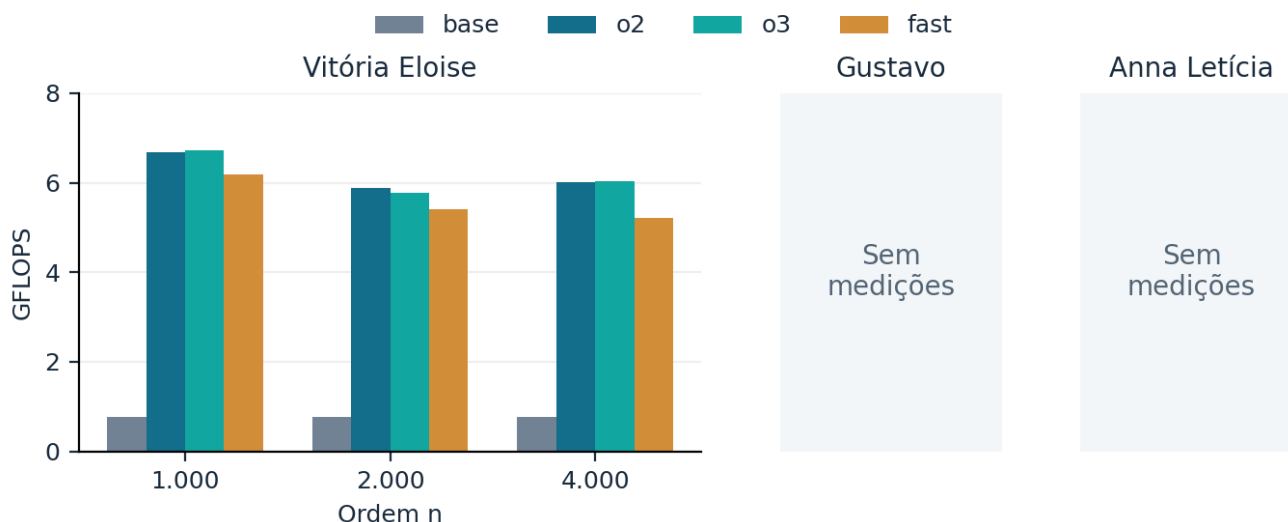


Figura 2. GFLOPS por alvo e ordem, com identificação das três máquinas. Apenas Eloise possui medições publicadas; os painéis sem barras representam ausência de dados, e não desempenho zero. Não é possível estabelecer um ranking entre CPUs. Fonte: [1].

6.1 Efeito das opções de compilação

A passagem de base para o2 reduz o tempo em fatores de 8,79x, 7,65x e 7,77x nas ordens 1.000, 2.000 e 4.000. O maior ganho está nessa primeira transição para otimização. Entre o2 e o3, as diferenças de tempo são de aproximadamente 0,44%, 2,09% e 0,26%; em 2.000, o2 é o mais rápido. Com um único registro por caso, não há evidência para afirmar superioridade geral de o3.

Ordem n	Melhor alvo registrado	Tempo extra de fast vs. o3	Ganho fast vs. base
1.000	o3	8,47%	8,14x
2.000	o2	6,76%	7,01x
4.000	o3	15,79%	6,73x

fast melhora muito em relação a base, mas perde para o2 e o3 em todos os tamanhos. Seu assembly publicado contém FMA escalar e não demonstra processamento SIMD de múltiplas colunas no núcleo analisado. Isso é compatível com a ausência de um ganho vetorial amplo, mas não explica isoladamente a regressão: faltam os assemblies de o2/o3, relatórios do compilador e repetições controladas.

6.2 Escala do problema e custo relativo de E/S

Ao passar de 1.000 para 4.000, o trabalho aritmético cresce 64x. O tempo de base cresce cerca de 62,82x, enquanto o de fast cresce 76,05x; seus GFLOPS caem de 6,1940 para 5,2126. Alterações de cache e frequência são hipóteses a testar, não causas demonstradas. No fast, a fração $\text{TEMPO_IO} / (\text{TEMPO_IO} + \text{TEMPO_COMP})$ cai de 1,54% para 0,43%. O cálculo domina essa soma medida, que exclui a inicialização de C.

7. Código de máquina: Vitória Eloise

Os arquivos com extensão .s são listagens de desassemblagem em sintaxe Intel, com endereços e bytes de máquina. A seleção abaixo conserva endereços e instruções do núcleo; omite bytes e usa reticências para trechos intermediários. Os nomes dos arquivos identificam a máquina, mas não há registro dos comandos exatos de compilação associados a cada listagem. [1]

asm_base	asm_fast
18c8: movss xmm0,DWORD PTR [rax] 18cc: mulss xmm0,DWORD PTR [rbp-0xd4] ... 18fc: addss xmm0,xmm1 1900: movss DWORD PTR [rax],xmm0 1904: add QWORD PTR [rbp-0x80],0x1 ... 1914: jb 1874 <main+0x583>	1720: vmovss xmm13, DWORD PTR [rdx+rdi*4] 1725: add rax,0x4 1729: inc rdi 172c: vfmadd213ss xmm13,xmm1, DWORD PTR [rax-0x4] 1732: vmovss DWORD PTR [rax-0x4],xmm13 1737: cmp rcx,rdi 173a: jne 1720 <main+0x500>

Trechos: maquinas/eloise/asm/asm_base_eloise.s, linhas 562-585; asm_fast_eloise.s, linhas 472-478. As quebras de operandos no quadro servem apenas à diagramação.

7.1 De operações separadas para FMA escalar

Em base, mulss multiplica um float e addss soma o resultado ao acumulador. O sufixo ss significa scalar single-precision: usar um registrador XMM não torna a operação vetorial. O entorno recalcula endereços com imul e lea, acessa variáveis na pilha e atualiza o índice em memória, comportamento coerente com uma compilação sem otimizações.

Em fast, vfmadd213ss combina multiplicação e adição. Na sintaxe Intel, o trecho realiza $xmm13 = xmm13 \times xmm1 + \text{memória}$, correspondente à atualização de um elemento de C. O avanço de rax em 4 bytes e o incremento unitário de rdi confirmam uma coluna por iteração. O fator $A[i*K+k]$ é carregado antes do laço e reutilizado em xmm1.

O arquivo não apresenta operações packed de 256 bits no núcleo analisado, embora as flags da CPU incluam AVX2 e FMA. Também não há desenrolamento visível desse laço interno: cada passagem atualiza um float. Não se deve afirmar que -funroll-loops obrigatoriamente desenrolou o núcleo apenas porque a opção consta no Makefile.

7.2 Interpretação dos resultados

A redução de trabalho de indexação, o uso de registradores e a FMA ajudam a interpretar o ganho de fast sobre base. Entretanto, a combinação de instruções não fornece uma medida direta de tempo, e não prova a causa da perda para o3. A listagem pode ter sido produzida sob opções ou uma revisão diferentes; não há vínculo documental entre seu binário e o CSV de desempenho.

FMA produz o produto e a soma com um único arredondamento final. Isso pode gerar resultados distintos da multiplicação e adição separadas. Como o projeto usa float e fast relaxa regras de ponto flutuante, a validação numérica deve considerar tolerâncias absoluta e relativa, além da velocidade. Não foram encontrados resultados publicados dessa validação.

8. Código de máquina: Gustavo

asm_base	asm_fast
<pre> 1740: movss xmm0,DWORD PTR [rax] 1744: mulss xmm0,DWORD PTR [rbp-0xd4] ... 1774: addss xmm0,xmm1 1778: movss DWORD PTR [rax],xmm0 177c: add QWORD PTR [rbp-0x80],0x1 ... 178c: jb 16ec <main+0x3fb> </pre>	<pre> 1607: vbroadcastss ymm1,xmm2 ... 1620: vmovups ymm0, YMMWORD PTR [rcx+rdx*1] 1625: vfmadd213ps ymm0,ymm1, YMMWORD PTR [rax+rdx*1] 162b: vmovups YMMWORD PTR [rax+rdx*1],ymm0 1630: add rdx,0x20 1634: cmp rdx,rsi 1637: jne 1620 <main+0x420> </pre>

Trechos: `maquinas/gustavo/asm/asm_base_gustavo.s`, linhas 474-497; `asm_fast_gustavo.s`, linhas 386-396. Reticências indicam omissões; operandos longos foram quebrados visualmente.

8.1 Vetorização efetiva do laço interno

O base de Gustavo também executa multiplicação e adição escalares separadas. No fast, `vbroadcastss` replica $A[i \cdot K + k]$ nas oito posições de `ymm1`. `vmovups` carrega oito floats de `B`, `vfmadd213ps` realiza oito atualizações de `C` e a instrução seguinte grava o vetor. O incremento de `0x20` equivale a 32 bytes, isto é, oito colunas por iteração.

O sufixo `ps` significa `packed single-precision`, em contraste com `ss`. Uma FMA de 256 bits realiza oito multiplicações e oito adições, contabilizadas como 16 operações aritméticas. Isso não significa automaticamente ganho de 8×: acessos à memória, controle dos laços, frequência e limites de execução também participam do tempo.

A listagem inclui testes de tamanho e de distância entre endereços antes da via vetorial, além de uma via escalar alternativa. Há tratamento de restos com XMM e instruções escalares. Como as três ordens solicitadas são múltiplas de oito, o resto por largura é zero quando a via YMM é selecionada. `vmovups` permite acessos sem exigência de alinhamento, mas sua presença não prova que os endereços estejam desalinhados.

8.2 Extensões presentes versus instruções utilizadas

Aspecto	Eloise: fast	Gustavo: fast
FMA observada no núcleo	<code>vfmadd213ss</code>	<code>vfmadd213ps</code>
Elementos por atualização mostrada	1 float	8 floats
Registrador da operação	XMM escalar	YMM de 256 bits
AVX-512 na CPU registrada	Não consta	Sim; sem ZMM no núcleo
Benchmark disponível	Sim	Não

O suporte de Gustavo a AVX-512 não obriga o GCC a gerar vetores de 512 bits: a listagem usa YMM. A diferença entre os arquivos evidencia estratégias distintas de geração de código, mas não permite atribuí-las exclusivamente à microarquitetura nem afirmar qual máquina termina primeiro. Para Anna Letícia, faltam CPU, assembly e tempos; nenhuma instrução ou desempenho pode ser inferido a partir dos outros integrantes.

9. Síntese e referências

9.1 Conclusões sustentadas pelas evidências

A implementação reúne condições favoráveis ao acesso regular à memória: blocos contíguos, alinhamento e percurso por linhas no laço interno. Nos testes de Eloise, a otimização reduz fortemente o tempo frente a base; o3 atinge o maior GFLOPS publicado, mas fast não vence em nenhuma ordem. O assembly confirma que FMA e SIMD são propriedades diferentes: Eloise apresenta FMA escalar, e Gustavo, FMA sobre oito floats.

A comparação entre CPUs permanece inconclusiva por ausência de tempos de Gustavo e Letícia. Também não é possível quantificar a vantagem de CSV ou BIN: só há E/S binária agregada. Logo, o relatório atende à análise dos materiais existentes, mas o conjunto experimental ainda precisa ser completado para cobrir integralmente os quatro itens do enunciado.

9.2 Dados necessários para fechar a análise

Coleta: publicar as quatro medidas de E/S para $n = 1.000, 2.000$ e 4.000 ; benchmarks dos quatro alvos em Gustavo e Letícia; e especificações e assemblies de Letícia. **Rastreabilidade:** registrar commit, comando completo, versão do GCC, identificação do binário e origem dos dados. **Comparabilidade:** usar as mesmas entradas, repetir execuções e informar mediana e dispersão, cache/aquecimento, afinidade e condições de energia. **Corretude:** confrontar as saídas com uma referência e tolerâncias explícitas. Relatórios de vetorização do GCC e assemblies de o2/o3 ajudariam a investigar a regressão de fast.

Referências e localização das evidências

[1] ELOISERXAZ et al. **Projeto_Implementacao_Produto_Interno**. Commit a88300a1bf3ef69351fd34d373347b8bbfec31b8. [Repositório na revisão analisada](#). Arquivos: src/matmul.c, src/csv2bin.c, src/bin2csv.c, Makefile, maquinas/eloise/benchmark_eloise.csv, benchmark_resultado.txt; cpu_info, ram_info e gcc_info em maquinas/eloise/specs e maquinas/gustavo/specs; asm_base e asm_fast em maquinas/eloise/asm e maquinas/gustavo/asm.

[2] AMD. **AMD Ryzen 5 5500: Specifications**. [Ficha oficial: microarquitetura e frequências nominais](#).

[3] INTEL. **Intel Core i5-11300H Processor: Specifications**. [Ficha oficial: Tiger Lake, frequência configurável e turbo](#).

[4] GNU PROJECT. **GCC 15.2: Optimize Options**. [Opções -O0, -O2, -O3, -ffast-math e -funroll-loops](#).

[5] GNU PROJECT. **Other Builtins**. [Semântica de __builtin_assume_aligned](#).

[6] GNU PROJECT. **x86 Options**. [Semântica de -march=native](#).

[7] INTEL. **Press Kit: 11th Gen Launch**. [Relação entre Tiger Lake e a microarquitetura Willow Cove](#).

Todas as fontes foram consultadas em 21/09/2026. As tabelas derivadas usam os valores do repositório; percentuais e ganhos foram calculados a partir dos tempos publicados. Nenhum dado experimental ausente foi estimado.